

Software de Orquestación

Terraform / OpenTofu

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 PI · Puesta en Producción de Aplicaciones

PI · SOFTWARE DE ORQUESTACIÓN

01

Infraestructura como Código

Qué es y por qué la usamos

Del despliegue manual al código

DESPLIEGUE TRADICIONAL

- Configuración manual, servidor a servidor
- **Difícil de reproducir** y de documentar
- La documentación se desactualiza o no existe
- *"Funciona en mi máquina"*

CON INFRAESTRUCTURA COMO CÓDIGO

- La infraestructura se **describe en ficheros de texto**
- Se crea y se modifica de forma **automática**
- Es **reproducible**: el mismo código da siempre el mismo resultado
- Se gestiona con **Git**, igual que el software

¿Qué es la Infraestructura como Código (IaC)?

" *Consiste en **gestionar y aprovisionar infraestructura** (servidores, redes, almacenamiento...) mediante **ficheros de código**, en lugar de configurarla manualmente.*

LO HACE POSIBLE

Los servicios de virtualización y Cloud Computing son **software**, y todo software se puede **programar mediante una API**.

BENEFICIOS

- Escenarios **replicables** y predecibles
- **Control de versiones** de toda la infraestructura
- **Auditoría** completa de los cambios
- Recuperación ante desastres mucho más rápida

Dos tipos de herramientas IaC



SOFTWARE DE ORQUESTACIÓN

Crea y gestiona **escenarios completos**:
máquinas virtuales, redes, almacenamiento...

Ejemplos: Terraform, OpenTofu, Pulumi,
CloudFormation, Heat



SOFTWARE DE GESTIÓN DE LA CONFIGURACIÓN (CMS)

Configura el **software** de las máquinas ya
creadas.

Ejemplos: Ansible, Puppet, Chef, Salt



En este módulo usamos **OpenTofu** para orquestar y **Ansible** para configurar: primero se crea la infraestructura, después se configura.

DevOps

Conflicto histórico: los equipos de Desarrollo (Dev) y de Sistemas (Ops) han tenido objetivos y herramientas distintas.

¿CÓMO LO SOLUCIONA DEVOPS?

- Mismas herramientas para dev y ops
- Extiende las buenas prácticas de desarrollo (Git, revisión de código, tests) a los sistemas
- Integración → entrega → despliegue **continuos**

¿TIENE RELACIÓN CON IAC?

Sí, es una de sus piedras angulares.
Sin IaC no se puede automatizar el aprovisionamiento de infraestructura, y sin automatización no hay DevOps.

PI · SOFTWARE DE ORQUESTACIÓN

02

Software de Orquestación

Crear infraestructura como código

¿Qué hace un software de orquestación?

Crea escenarios completos con múltiples servidores, redes o contenedores (aprovisionamiento de recursos).

- Útil con demanda **variable** de recursos
- Útil cuando la configuración **cambia continuamente**
- Puede incluir **autoescalado** y respuesta a eventos

ENFOQUE DECLARATIVO

En lugar de describir *cómo* crear la infraestructura, **declaramos el estado deseado** y la herramienta se encarga de alcanzarlo.

```
resource "libvirt_domain" "web" {  
  name    = "servidor-web"  
  memory = 1024  
  vcpu    = 1  
}
```

PI · SOFTWARE DE ORQUESTACIÓN

03

OpenTofu

Orquestación de infraestructura declarativa

De Terraform a OpenTofu

TERRAFORM

Herramienta de orquestación creada por **HashiCorp** (2014), lenguaje declarativo propio **HCL**. En agosto de 2023 cambió su licencia a **BUSL 1.1**, que ya no es software libre.

OPENTOFU

Fork libre de Terraform 1.5.x creado por la comunidad y gobernado por la **Linux Foundation**. Licencia **MPL 2.0** (libre), 100% compatible con los ficheros `.tf` y providers existentes.



Usamos **OpenTofu** en lugar de Terraform precisamente por ser software libre, sin restricciones de uso.

Elementos de configuración

LOS CUATRO BLOQUES BÁSICOS

- **Provider:** plugin que conecta con una API concreta
- **Resource:** elemento de infraestructura a crear (VM, red, volumen...)
- **Variable:** parámetro para reutilizar la configuración
- **Output:** valor de salida (IP, ID...) tras aplicar el plan

EJEMPLO

```
resource "aws_instance" "web" {  
    ami          = var.imagen  
    instance_type = "t2.micro"  
}  
  
output "ip_publica" {  
    value = aws_instance.web.public_ip  
}
```

Un mismo lenguaje, distintos providers

Un **provider** es un plugin que conecta OpenTofu con una plataforma concreta. Cada uno define sus propios *resources*, pero el lenguaje (HCL) es siempre el mismo — en este módulo usaremos `libvirt` y `openstack`.

PROVIDER	PLATAFORMA
<code>aws</code>	Amazon Web Services
<code>azurerm</code>	Microsoft Azure
<code>google</code>	Google Cloud Platform
<code>openstack</code>	Nube privada OpenStack
<code>libvirt</code>	Máquinas virtuales con KVM
<code>docker</code>	Contenedores Docker

Ficheros más importantes de un proyecto

LOS ESCRIBIMOS NOSOTROS

- `provider.tf` — configura el/los provider a usar
- `variables.tf` — declara las variables reutilizables
- `main.tf` — define los recursos: VMs, redes, discos...
- `output.tf` — muestra información al finalizar (IPs, IDs...)

LO GENERA OPENTOFU

- `terraform.tfstate` — el **estado** de la infraestructura
- `.terraform/` — providers y módulos descargados por `init`

No se editan a mano.

El estado de la infraestructura

" El **estado** (`terraform.tfstate`) es un fichero donde OpenTofu guarda **qué recursos ha creado realmente** y con qué identificadores (IDs, IPs, dependencias...).

¿POR QUÉ SE GUARDA?

- Es la única forma de saber **qué existe ya** sin preguntar a la plataforma recurso a recurso
- Permite **comparar** lo declarado en el código con lo que hay realmente creado
- Sin él, OpenTofu no sabría qué modificar o eliminar

CUIDADO CON EL ESTADO

- **No se edita a mano**
- Si varios usuarios trabajan a la vez, hace falta un **backend remoto** compartido para evitar conflictos
- Si el estado se pierde, OpenTofu "olvida" lo que había creado

Comandos más útiles

COMANDO	DESCRIPCIÓN
<code>tofu init</code>	Inicializa el proyecto, descarga los <i>providers</i>
<code>tofu plan</code>	Muestra qué acciones se realizarán (sin aplicar)
<code>tofu apply</code>	Aplica los cambios: crea, modifica o elimina recursos
<code>tofu destroy</code>	Elimina todos los recursos gestionados por el proyecto
<code>tofu validate</code>	Verifica que la sintaxis de los ficheros <code>.tf</code> sea correcta
<code>tofu show</code>	Muestra el estado actual de los recursos creados
<code>tofu output</code>	Muestra los valores definidos en los bloques <code>output</code>

¿Qué hace realmente `tofu plan`?

" `plan` *compara tres cosas y muestra la diferencia, sin modificar nada todavía.*

1. EL CÓDIGO

Lo que **declaras** en los ficheros `.tf` (el estado deseado)

2. EL ESTADO

Lo último que OpenTofu **sabe** que existe
(`terraform.tfstate`)

3. LA REALIDAD

Lo que **hay de verdad** en la plataforma (puede haber cambiado a mano: *drift*)



`plan` es la red de seguridad antes de aplicar: revisa siempre su salida antes de ejecutar `apply`.

PI · SOFTWARE DE ORQUESTACIÓN

¡Gracias!

Software de Orquestación · OpenTofu



José Domingo Muñoz



IES Gonzalo Nazareno · Dos Hermanas



PI